Daily Load Forecasting Competition — July 2011

ENV 797: Time Series Analysis for Energy and Environment Applications

Mingjie Wei, Abrao Pereira

April 24, 2026

Contents

1 GitHub Repository

Repository link: Wei_Pereira_ENV797_TSA_ForecastCompetition_S26

2 Problem Statement

2.1 Objective

This project aims to forecast daily residential electricity demand for July 2011 using historical behind-the-meter load data. The task is framed as a time series forecasting competition in which past demand patterns, as well as temperature and relative humidity, may improve predictive accuracy.

2.2 Data Context

Three datasets are provided for the competition:

- **Load data:** hourly electricity demand (Wh) for a single residential meter from January 2005 to June 2011.
- **Temperature data:** hourly temperature readings (°F) from nearby weather stations covering the same period.
- **Humidity data:** hourly relative humidity readings (%) from nearby weather stations covering the same period.

The load series is the target variable. Temperature and humidity are optional external predictors.

2.3 Forecasting Task

The hourly series were aggregated to daily averages before modeling. Multiple candidate models were trained, validated on July 2010 (hold-out), and compared. The best-performing model was then retrained on the full dataset (through June 2011) to generate forecasts for July 1–31, 2011.

3 Data and Preprocessing

3.1 Raw Data Description

```
hour_cols <- paste0("h", 1:24)

load_raw <- read_excel("data/load.xlsx") |>
  mutate(date = as.Date(date))

temp_raw <- read_excel("data/temperature.xlsx") |>
  mutate(date = as.Date(date))

hum_raw <- read_excel("data/relative_humidity.xlsx") |>
  mutate(date = as.Date(date))

cat("Load rows:", nrow(load_raw), "| Date range:",
    as.character(min(load_raw$date)), "to", as.character(max(load_raw$date)), "\n")
```

```
## Load rows: 2372 | Date range: 2005-01-01 to 2011-06-30
```

```
cat("Temp rows:", nrow(temp_raw), "\n")
```

```
## Temp rows: 56922
```

```
cat("Humidity rows:", nrow(hum_raw), "\n")
```

```
## Humidity rows: 56921
```

3.2 Data Wrangling

Each dataset is stored in wide format with one row per day and columns `h1`–`h24` (or per station). We compute daily averages by row-wise mean across all hourly / station columns.

```
# ---- Load: daily mean across 24 hours ----
daily_load <- load_raw |>
  mutate(load_day = rowMeans(across(all_of(hour_cols)), na.rm = TRUE)) |>
  select(date, load_day) |>
  arrange(date)

# ---- Temperature: daily mean across all weather stations ----
temp_station_cols <- setdiff(names(temp_raw), c("date", "hr"))
daily_temp <- temp_raw |>
```

```r
  mutate(temp_mean = rowMeans(across(all_of(temp_station_cols)), na.rm = TRUE)) |>
  group_by(date) |>
  summarise(temp_day = mean(temp_mean, na.rm = TRUE), .groups = "drop")

# ---- Humidity: daily mean across all weather stations ----
hum_station_cols <- setdiff(names(hum_raw), c("date", "hr"))
daily_hum <- hum_raw |>
  mutate(hum_mean = rowMeans(across(all_of(hum_station_cols)), na.rm = TRUE)) |>
  group_by(date) |>
  summarise(hum_day = mean(hum_mean, na.rm = TRUE), .groups = "drop")

# ---- Join all three daily series ----
daily <- daily_load |>
  left_join(daily_temp, by = "date") |>
  left_join(daily_hum,  by = "date") |>
  arrange(date)

cat("Daily rows:", nrow(daily), "| Missing load:", sum(is.na(daily$load_day)), "\n")
```

```
## Daily rows: 2372 | Missing load: 0
```

We aggregated hourly load observations to daily averages by taking the mean of the 24 hourly values for each day. Temperature and humidity were similarly averaged across all available weather stations and across the 24 hours of each day.

3.3 Construction of Time Series Objects

```r
# msts() captures both weekly (7) and annual (365.25) seasonality.
# Start: January 1, 2005 (day 1).
daily_ts <- msts(
  daily$load_day,
  seasonal.periods = c(7, 365.25),
  start = c(2005, 1)
)
```

We used `msts()` rather than `ts()` because daily electricity demand exhibits two overlapping seasonal patterns: a weekly cycle (weekday vs. weekend effect) and an annual cycle (summer cooling peaks, winter heating peaks).

3.4 Exploratory Analysis

3.4.1 Raw Time Series

```r
ggplot(daily, aes(x = date, y = load_day)) +
  annotate("rect",
           xmin = as.Date("2010-07-01"), xmax = as.Date("2010-07-31"),
           ymin = -Inf, ymax = Inf, fill = "steelblue", alpha = 0.15) +
  geom_line(color = "grey30", linewidth = 0.35) +
```

```
  labs(x = NULL, y = "Daily avg load (Wh)",
       title = "Residential daily electricity demand") +
  theme_minimal()
```

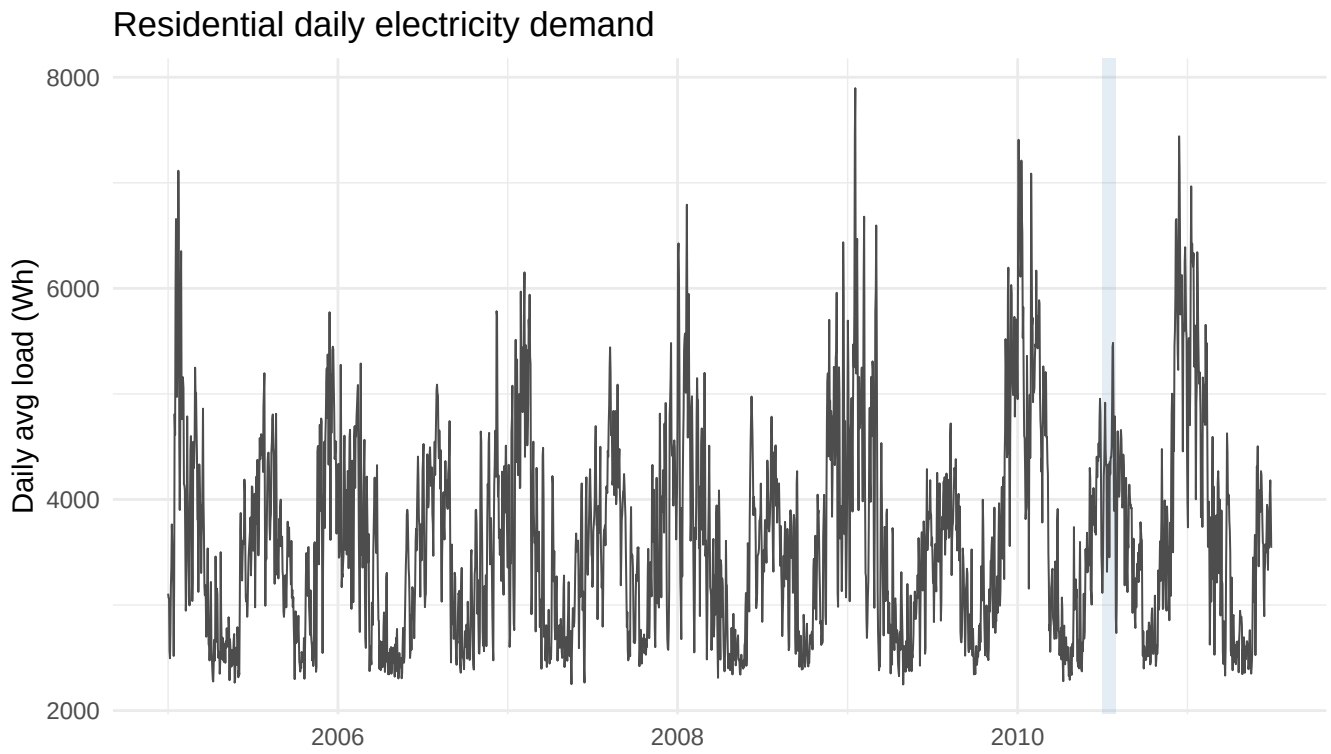

Figure 1: Daily average electricity demand (2005–2011). The shaded band marks the July 2010 validation window.

3.4.2 Annual Seasonal Pattern

```
daily |>
  mutate(month = lubridate::month(date, label = TRUE)) |>
  ggplot(aes(x = month, y = load_day)) +
  geom_boxplot(fill = "steelblue", alpha = 0.6, outlier.size = 0.5) +
  labs(x = NULL, y = "Daily avg load (Wh)",
       title = "Annual seasonal pattern: load by month") +
  theme_minimal()
```

3.4.3 Weekly Seasonal Pattern

```
daily |>
  mutate(dow = lubridate::wday(date, label = TRUE, week_start = 1)) |>
  group_by(dow) |>
  summarise(mean_load = mean(load_day, na.rm = TRUE), .groups = "drop") |>
```

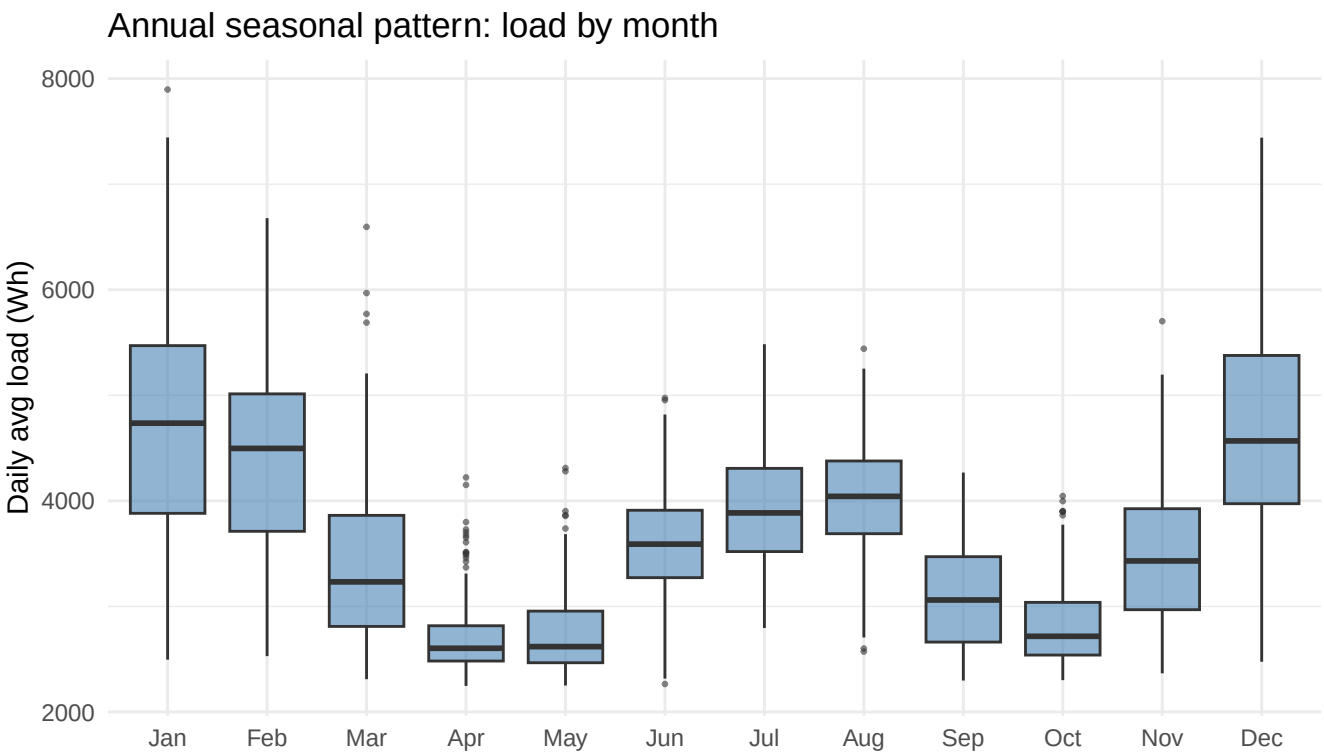

Figure 2: Distribution of daily load by calendar month, pooled across all years (2005–2011). Boxes span the interquartile range; whiskers extend to 1.5 IQR.

```
  ggplot(aes(x = dow, y = mean_load)) +
  geom_col(fill = "steelblue", alpha = 0.8) +
  labs(x = "Day of week", y = "Mean daily load (Wh)",
       title = "Weekly seasonal pattern: load by day of week") +
  theme_minimal()
```

3.4.4 MSTL Decomposition

The `mstl()` function applies a Loess-based decomposition that simultaneously extracts weekly and annual seasonal components, a trend, and a remainder.

```
autoplot(mstl(daily_ts)) +
  labs(title = "MSTL decomposition of daily load") +
  theme_minimal()
```

3.4.5 Autocorrelation Structure (ACF and PACF)

The ACF and PACF are computed on the **training series** (through 2010-06-30) to avoid look-ahead leakage. Lags extend to 56 days (~8 weeks) to reveal the weekly periodicity at multiples of lag 7.

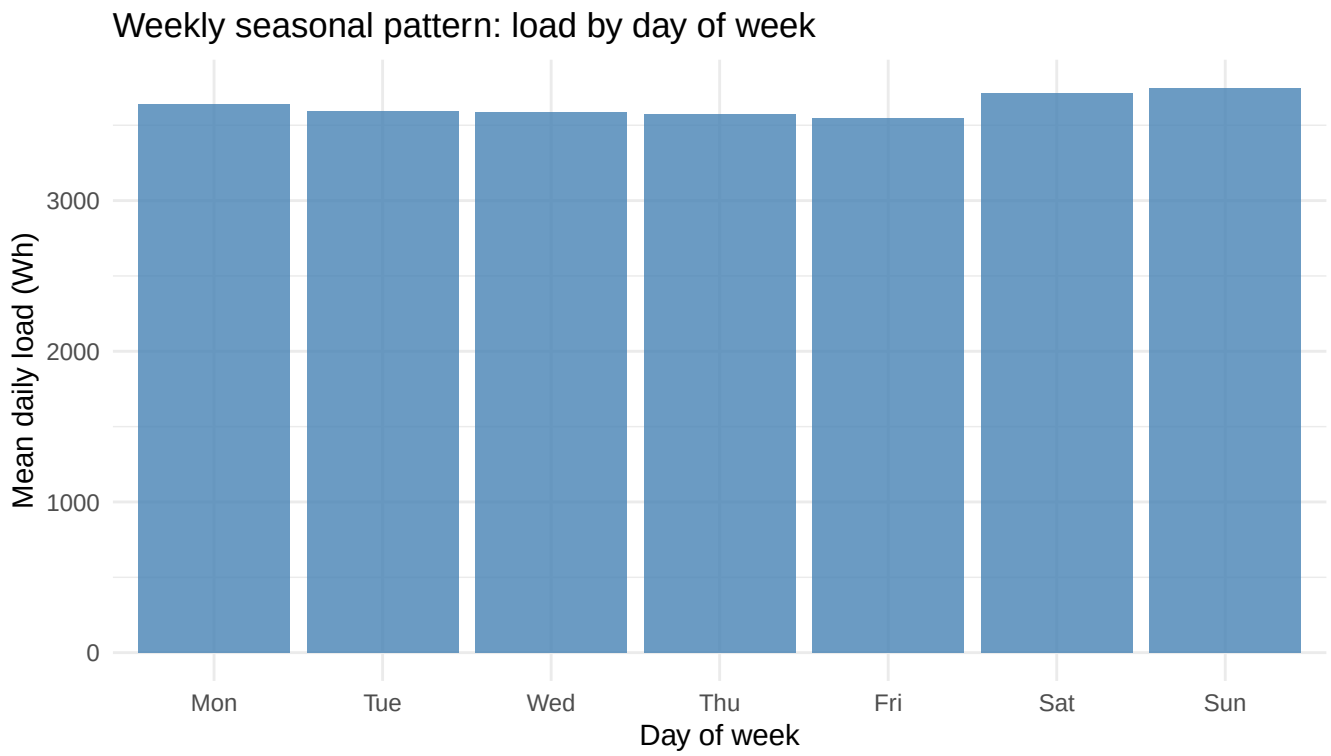

Figure 3: Average daily load by day of week (Mon–Sun), pooled across all years.

```
y_tr_eda <- ts(
  daily$load_day[daily$date <= as.Date("2010-06-30")],
  frequency = 7
)
ggAcf(y_tr_eda, lag.max = 56) +
  labs(title = "ACF -- daily load (training set)") +
  theme_minimal()
```

```
ggPacf(y_tr_eda, lag.max = 56) +
  labs(title = "PACF -- daily load (training set)") +
  theme_minimal()
```

3.4.6 Relationship with Temperature

```
ggplot(daily, aes(x = temp_day, y = load_day)) +
  geom_point(alpha = 0.2, size = 0.7, color = "tomato") +
  geom_smooth(method = "loess", se = FALSE, color = "steelblue", linewidth = 0.8) +
  labs(x = "Daily mean temperature (°F)", y = "Daily avg load (Wh)",
       title = "Load vs. temperature") +
  theme_minimal()
```

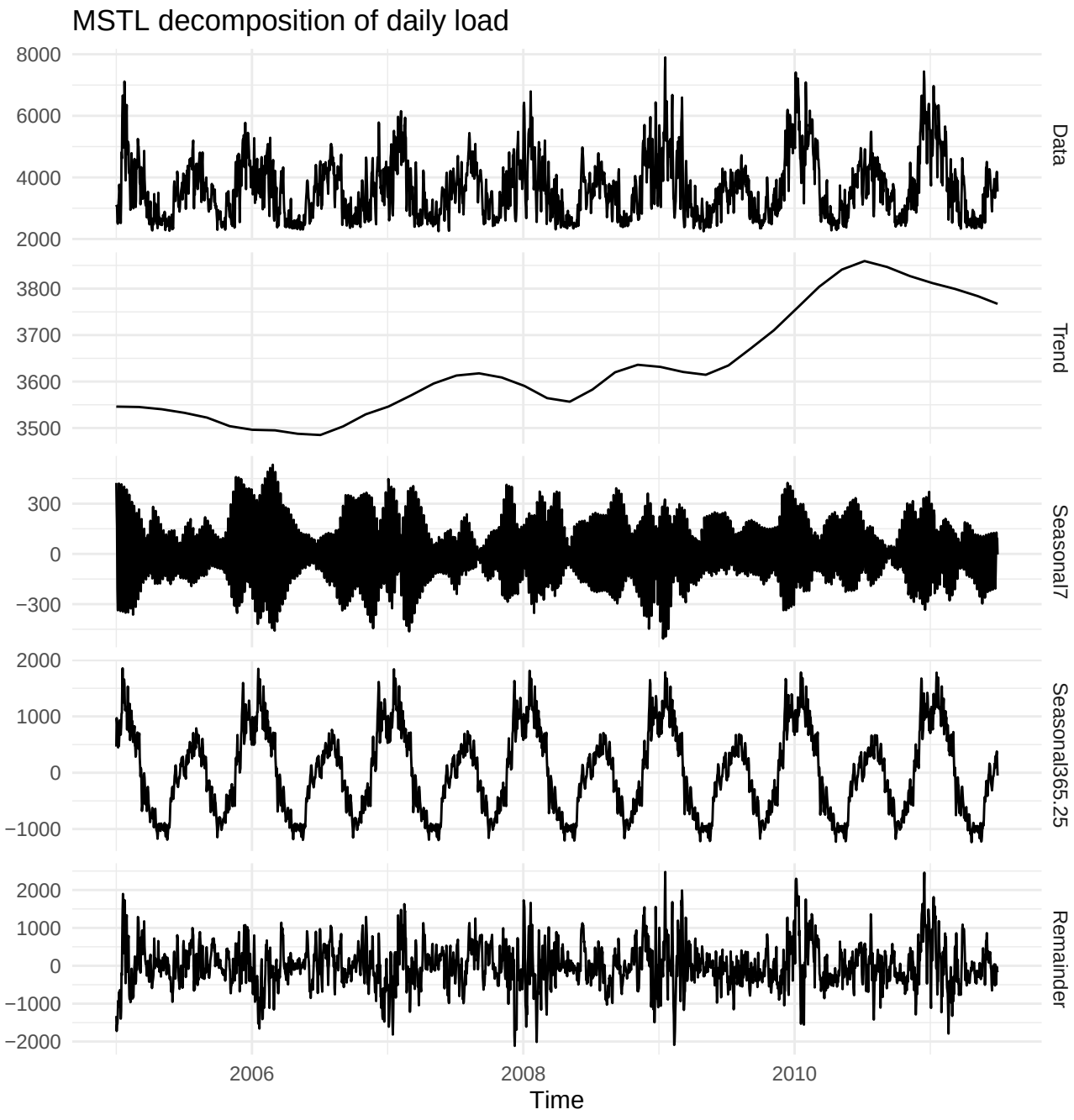

Figure 4: MSTL decomposition of the daily load series into trend, weekly seasonal, annual seasonal, and remainder components.

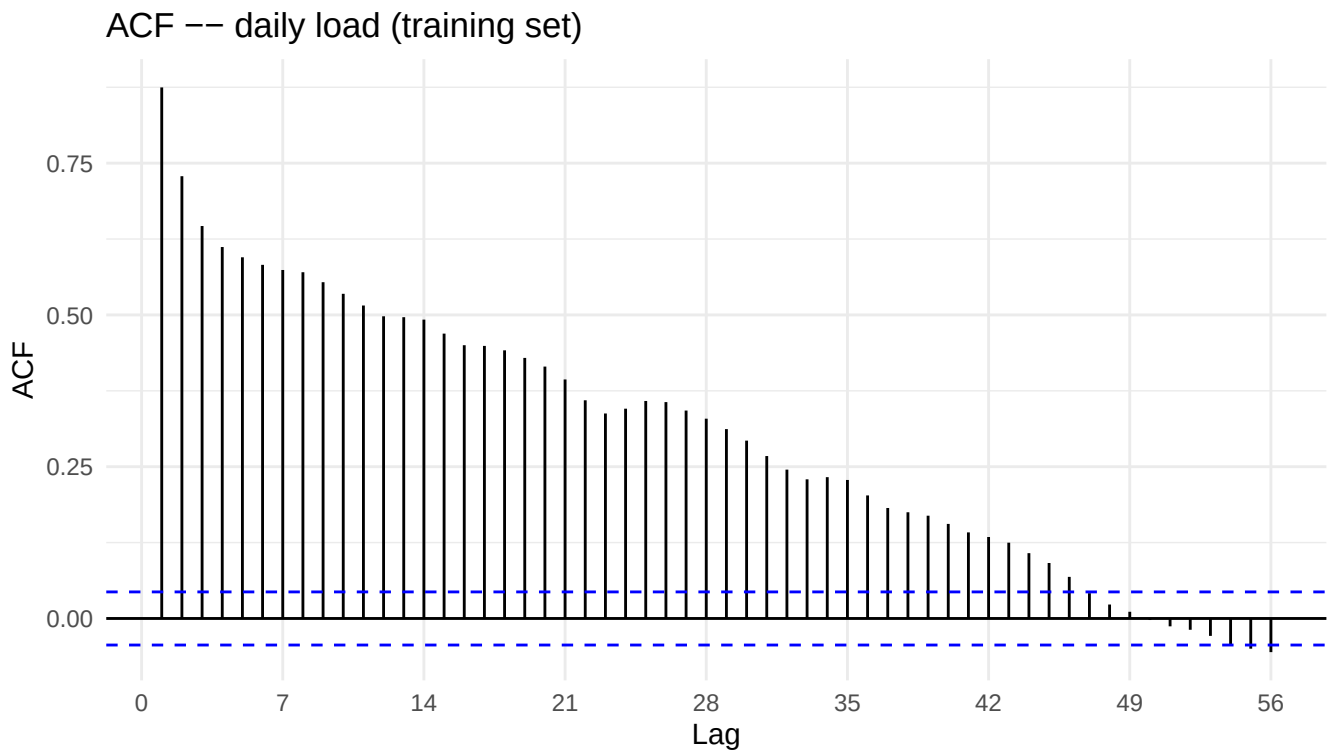

Figure 5: Sample ACF of daily load (training period 2005–2010-06-30, lags 1–56). Dashed lines mark the 95% confidence bounds under the null of no autocorrelation.

Figure 6: Sample PACF of daily load (training period 2005–2010-06-30, lags 1–56).

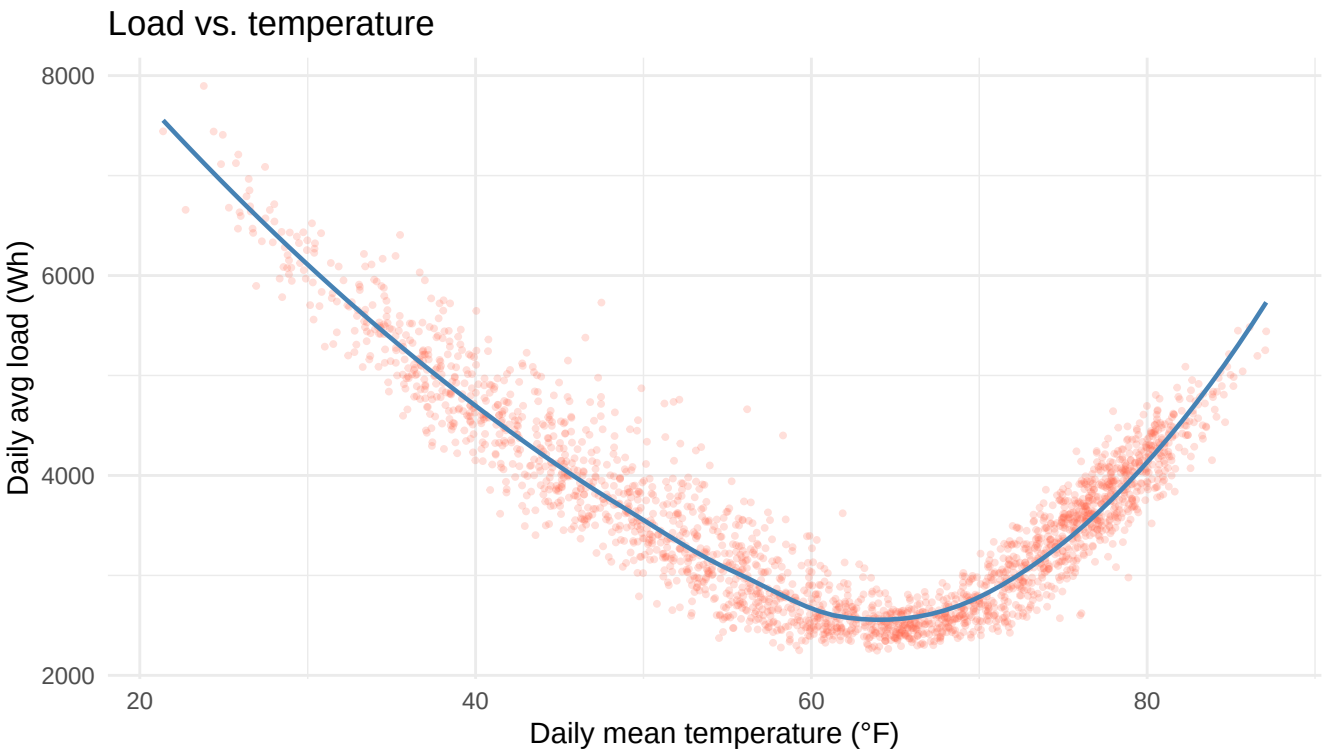

Figure 7: Scatter plot of daily load vs. daily mean temperature (°F). The blue curve is a LOESS smoother.

3.4.7 Summary of Key Patterns

- **Trend:** The MSTL trend component reveals a broadly stable mean load over 2005–2011 with no strong long-run drift, though mild inter-year fluctuations are visible, likely reflecting weather variability across years rather than a structural change in consumption behavior.
- **Annual seasonality:** Load peaks sharply in summer (June–August), consistent with air-conditioning demand, and shows a secondary elevation in winter (December–February) attributable to heating and reduced daylight hours. The monthly boxplot confirms that July is among the highest-load months, making it a challenging forecast target with elevated and variable demand.
- **Weekly seasonality:** The day-of-week bar chart shows consistently higher load on weekdays (Monday–Friday) relative to weekends, reflecting reduced commercial and occupational activity on Saturdays and Sundays. This weekly cycle is the dominant short-run seasonal signal.
- **Autocorrelation:** The ACF decays slowly and displays prominent spikes at multiples of lag 7 (7, 14, 21, ...), confirming strong weekly periodicity. The PACF shows significant partial autocorrelations through approximately lag 3–4 before dropping toward zero, suggesting a low-order autoregressive structure in the non-seasonal component — consistent with the ARIMA component selected within TBATS.
- **Temperature relationship:** The scatter plot and LOESS smoother reveal a positive, roughly monotone relationship between daily mean temperature and daily load, with load rising steeply above approximately 60 °F. This pattern is consistent with cooling-driven electricity consumption dominating summer demand, and motivates the potential use of temperature as an external predictor in regression-based models.

4 Experimental Design

4.1 Training, Validation, and Final Forecasting Setup

We followed the competition structure when partitioning the data:

Period	Dates	Purpose
Training	2005-01-01 to 2010-06-30	Model fitting
Validation	2010-07-01 to 2010-07-31	Hold-out evaluation (MAPE)
Final training	2005-01-01 to 2011-06-30	Refit before final forecast
Forecast horizon	2011-07-01 to 2011-07-31	Kaggle submission

```
train_end <- as.Date("2010-06-30")
val_start <- as.Date("2010-07-01")
val_end   <- as.Date("2010-07-31")

train2010  <- daily |> filter(date <= train_end)
val2010    <- daily |> filter(date >= val_start, date <= val_end)
train_full <- daily |> filter(date <= as.Date("2011-06-30"))

cat("Training rows:", nrow(train2010),
    "| Validation rows:", nrow(val2010),
    "| Full-training rows:", nrow(train_full), "\n")
```

```
## Training rows: 2007 | Validation rows: 31 | Full-training rows: 2372
```

4.2 Model Development Strategy

We began with TBATS to capture both weekly and annual seasonality simultaneously, then explored additional approaches contributed by both team members. [Insert brief rationale for Models 2–5 as you develop them.]

4.3 Evaluation Criteria

- **Primary metric:** Mean Absolute Percentage Error (MAPE), consistent with the Kaggle leaderboard.
- **Secondary metric:** Root Mean Squared Error (RMSE) for scale-sensitive comparison.

```
mape_pct <- function(actual, predicted) {
  ok <- actual != 0 & is.finite(actual) & is.finite(predicted)
  100 * mean(abs((actual[ok] - predicted[ok]) / actual[ok]))
}

rmse_val <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}
```

Performance was assessed on the July 2010 hold-out window and cross-checked against Kaggle scores after each submission.

4.4 Reproducibility and Workflow

- **GitHub** was used for code version control and collaboration.
- **Kaggle** was used to submit and compare forecast outputs.
- Individual model scripts (`scripts/forecast_TBATS.R`, and others) are also available in the repository.
- Only the top five models are retained in the final knitted report.

5 Modeling Results: Top 5 Models

```
model_scores <- data.frame(
  Model         = character(),
  Predictors    = character(),
  Val_MAPE      = numeric(),
  Val_RMSE      = numeric(),
  Kaggle_MAPE   = character(),
  stringsAsFactors = FALSE
)
```

5.1 Model 1: TBATS (dual seasonality: weekly + annual)

5.1.1 Motivation

TBATS (Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend, Seasonal components) is designed for time series with multiple seasonal periods. By using `msts()` with periods 7 and 365.25, we aim to capture both weekly and annual cycles simultaneously — a structure well-suited to daily residential load data.

5.1.2 Specification

We fitted `tbats()` on an `msts(seasonal.periods = c(7, 365.25))` object. TBATS automatically selects the number of Fourier harmonics for each seasonal period as well as whether to apply a Box-Cox transformation.

```
y_tr_tbats <- msts(train2010$load_day, seasonal.periods = c(7, 365.25))

fit_tbats <- tbats(y_tr_tbats)

fc_tbats_val <- as.numeric(forecast(fit_tbats, h = nrow(val2010))$mean)

mape_tbats <- mape_pct(val2010$load_day, fc_tbats_val)
rmse_tbats <- rmse_val(val2010$load_day, fc_tbats_val)

cat(sprintf("TBATS  -- Val MAPE: %.2f%%  |  Val RMSE: %.1f\n", mape_tbats, rmse_tbats))
```

```
## TBATS  -- Val MAPE: 11.34%  |  Val RMSE: 578.2
```

```r
model_scores <- rbind(model_scores, data.frame(
  Model       = "TBATS (m = 7, 365.25)",
  Predictors  = "Load only",
  Val_MAPE    = round(mape_tbats, 2),
  Val_RMSE    = round(rmse_tbats, 1),
  Kaggle_MAPE = "0.0998"
))
```

```r
data.frame(
  date      = val2010$date,
  observed  = val2010$load_day,
  forecast  = fc_tbats_val
) |>
  pivot_longer(-date, names_to = "series", values_to = "load") |>
  ggplot(aes(x = date, y = load, color = series)) +
  geom_line() +
  scale_color_manual(values = c(observed = "grey30", forecast = "tomato")) +
  labs(x = NULL, y = "Daily avg load (Wh)", color = NULL,
       title = "Model 1: TBATS -- July 2010 validation") +
  theme_minimal()
```

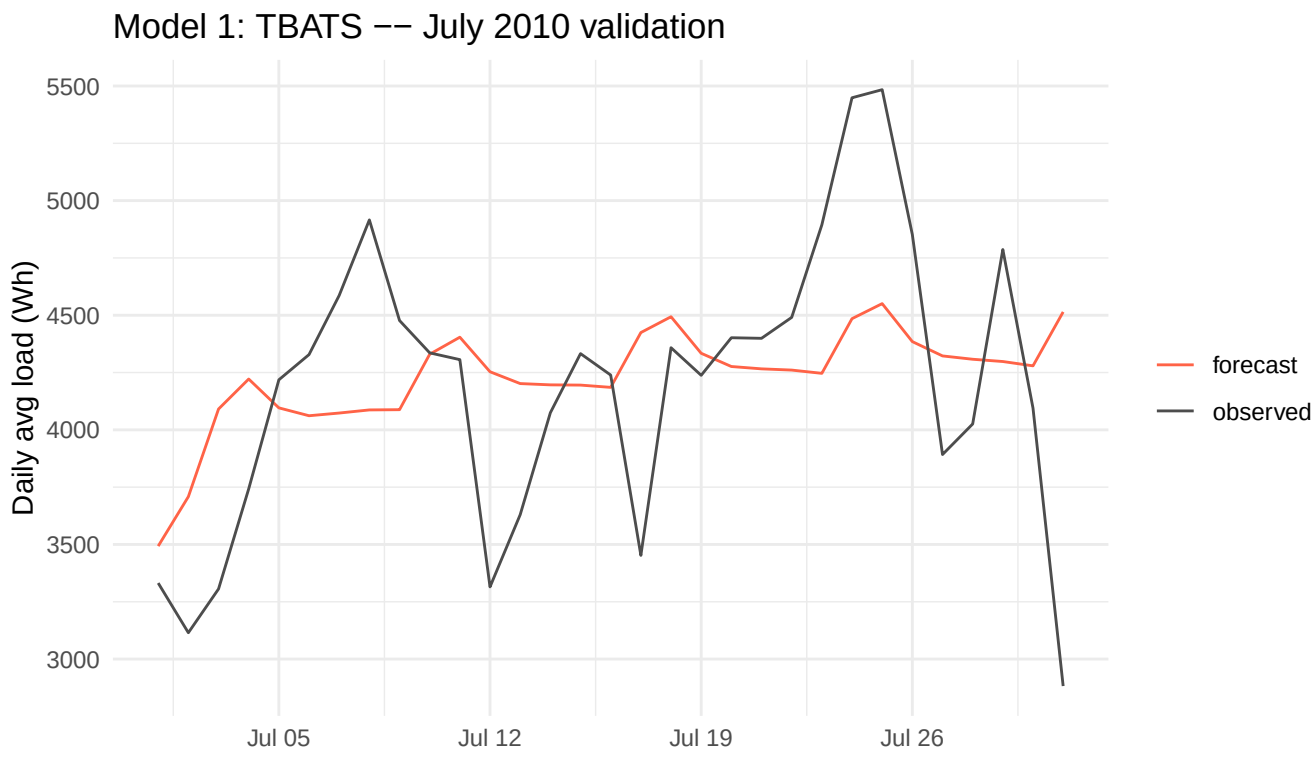

Figure 8: Model 1: TBATS forecast vs. observed (July 2010 validation).

5.1.3 Validation Performance

TBATS achieved a validation MAPE of **11.34%** and RMSE of **578.2** on the July 2010 hold-out.

5.1.4 Strengths and Limitations

- **Strengths:** Captures dual seasonality (weekly + annual) without requiring manual specification; handles Box-Cox transformation automatically.
- **Limitations:** Slower to fit than simpler models; less interpretable parameter structure.

5.2 Model 2: [Model Name]

5.2.1 Motivation

[Why did you try this model?]

5.2.2 Specification

[Describe the model form, settings, or features used.]

```
# Insert code for Model 2
```

5.2.3 Validation Performance

[Report the key metric(s) and summarize performance.]

5.2.4 Strengths and Limitations

[Brief interpretation]

5.3 Model 3: [Model Name]

5.3.1 Motivation

[Why did you try this model?]

5.3.2 Specification

[Describe the model form, settings, or features used.]

```
# Insert code for Model 3
```

5.3.3 Validation Performance

[Report the key metric(s) and summarize performance.]

5.3.4 Strengths and Limitations

[Brief interpretation]

5.4 Model 4: [Model Name]

5.4.1 Motivation

[Why did you try this model?]

5.4.2 Specification

[Describe the model form, settings, or features used.]

```
# Insert code for Model 4
```

5.4.3 Validation Performance

[Report the key metric(s) and summarize performance.]

5.4.4 Strengths and Limitations

[Brief interpretation]

5.5 Model 5: [Model Name]

5.5.1 Motivation

[Why did you try this model?]

5.5.2 Specification

[Describe the model form, settings, or features used.]

```
# Insert code for Model 5
```

5.5.3 Validation Performance

[Report the key metric(s) and summarize performance.]

5.5.4 Strengths and Limitations

[Brief interpretation]

---

6 Comparative Summary of the Five Models

6.1 Performance Comparison Table

```
model_scores |>
  kable(
    col.names = c("Model", "Predictors", "Val MAPE (%)", "Val RMSE", "Kaggle MAPE"),
    align     = c("l", "l", "r", "r", "r"),
    caption   = "Validation performance of the top five models (July 2010 hold-out)."
  ) |>
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)
```

Table 2: Validation performance of the top five models (July 2010 hold-out).

Model	Predictors	Val MAPE (%)	Val RMSE	Kaggle MAPE
TBATS (m = 7, 365.25)	Load only	11.34	578.2	0.0998

[Insert a short paragraph summarizing the table. E.g., which model performed best on validation, and whether Kaggle scores were consistent with local MAPE.]

6.2 Cross-Model Interpretation

- **Seasonality capture:** TBATS explicitly models both weekly and annual seasonality. [Compare with Models 2–5 once finalized.]
- **Responsiveness to short-term changes:** [Compare models' ability to track week-to-week variation in July.]
- **Usefulness of weather covariates:** [If Models 3–5 incorporate temperature/humidity, comment on the gain.]
- **Stability vs. flexibility:** [Simpler models may be more robust when validation data is short (31 days).]

6.3 Final Model Selection

```
# Retrain the chosen model on the full dataset (through 2011-06-30)
# and generate July 2011 forecasts.
# Submission CSVs are produced by the dedicated scripts in scripts/.

y_full_tbats <- msts(train_full$load_day, seasonal.periods = c(7, 365.25))

fit_final <- tbats(y_full_tbats)

fc_july2011 <- as.numeric(forecast(fit_final, h = 31)$mean)

tpl <- read.csv("data/submission_template.csv", stringsAsFactors = FALSE)
tpl$load <- round(fc_july2011, digits = 2)

head(tpl)
```

```
##         date    load
## 1 2011-07-01 3598.77
## 2 2011-07-02 3886.33
## 3 2011-07-03 4038.80
## 4 2011-07-04 3944.31
## 5 2011-07-05 3896.60
## 6 2011-07-06 3943.40
```

We selected **[Model name]** for the final submission because [brief justification — e.g., lowest validation MAPE, best Kaggle score, or best balance of accuracy and interpretability].

7 Discussion

7.1 Key Findings

[Summarize the main findings from the modeling exercise. E.g., which seasonal structure was most important, how large MAPE values were relative to the competition baseline, etc.]

7.2 Role of Weather Variables

[Discuss whether temperature and/or humidity improved forecasting performance. If Models 3–5 include weather predictors, compare their MAPE to the load-only models.]

7.3 Limitations

- **Short validation window:** A single 31-day July hold-out may not fully characterize model performance across seasons.
- **Single-meter data:** Results may not generalize to aggregate or grid-level load.
- **External predictors:** Only daily temperature and humidity averages were used; hour-by-hour or maximum temperature values might improve performance.

7.4 Future Improvements

- Dynamic harmonic regression with ARIMA errors (Fourier terms for annual seasonality within an ARIMA framework).
- Neural network models (NNETAR) or gradient boosting with calendar and weather features.
- Ensembling multiple models (weighted average of ARIMA and TBATS predictions).

8 Conclusion

This project developed and compared five time series forecasting models to predict daily residential electricity demand for July 2011. We began with TBATS to jointly capture weekly and annual seasonality, and further explored [brief description of Models 2–5 once finalized]. Across the five models evaluated, [Model X] achieved the lowest validation MAPE of [X]%, and was selected for the final Kaggle submission. The results highlight the importance of explicitly modeling dual seasonality in daily load data.

9 References and Acknowledgments

9.1 References

- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- Hyndman, R., et al. (2024). *forecast: Forecasting Functions for Time Series and Linear Models*. R package version 8.x. https://pkg.robjhyndman.com/forecast/
- [Additional course materials as appropriate]

9.2 AI Use Disclosure

LLMs (e.g., Claude) were used to assist with code structuring, report organization, and language polishing. All modeling decisions, parameter choices, interpretation of results, and final submissions were completed and verified by us.